Evaluating GPT-5.4-mini for Gherkin Test Generation: An Empirical Study

Nguyen Minh Hoang, Nguyen Hieu An, Tran Tuan Khang, Vo Hieu Chuong, Tong Vy Thuan, and Le Thi Quynh Chi*

FPT University, Vietnam
chiltq6@fe.edu.vn

Abstract. Large language models (LLMs) can reduce the effort of drafting behavior-driven development (BDD) scenarios, but valid syntax does not guarantee that business intent is preserved. We evaluate GPT-5.4-mini on Connextra-style user stories reverse-engineered from 261 expert-written feature files in Sylius, Apache Fineract and Diaspora, using one same-project example per prompt. We measure parser compliance with the behave Gherkin parser and similarity to the reference with skeleton cosine similarity (all-MiniLM-L6-v2). Excluding the three prompt exemplars ($N = 258$), 255 files (98.8%, 95% CI [96.6%, 99.8%]) are valid Gherkin, well above the 80% threshold ($p < 0.001$). The median similarity is 0.853, but a one-sided Wilcoxon test does not show that it exceeds the 0.85 threshold ($p = 0.474$). Only the e-commerce project exceeds it; the finance and social-network projects lie significantly below it, and the conclusion changes at a threshold of 0.84. Among 60 high-similarity files, 25 contain a business-logic defect in AI-drafted, human-reviewed labels (agreement with an independent rater: $\kappa = 0.57$), and seeded defects barely change the score. The similarity result also depends on the prompt: without an example the median falls to 0.819, and with a different exemplar per project it rises to 0.875, significantly above 0.85, while parser compliance stays at 98.8–99.6%. GPT-5.4-mini is therefore a drafting aid whose business logic needs human review, not an autonomous acceptance-test generator, and embedding similarity alone cannot certify generated Gherkin.

Keywords: Large language models · Gherkin · Behavior-driven development · Acceptance testing · Software testing · GPT-5.4-mini

1 Introduction

Behavior-driven development expresses expected software behavior in executable or executable-adjacent scenarios, commonly using Gherkin. In practice, the cost of BDD is not limited to writing Given–When–Then syntax. Engineers must preserve domain preconditions, project conventions, data dependencies, and expected outcomes while keeping feature files parsable by the project toolchain.

* Corresponding author.

A generated scenario can therefore be syntactically valid yet incomplete as a specification of business behavior.

The three projects in this study illustrate this distinction. A Sylius checkout scenario may depend on channel, country, shipping-method state, and confirmation behavior. Apache Fineract scenarios frequently encode dates, repayment periods, balances, and data tables. Diaspora scenarios emphasize browser-visible behavior such as navigation, tags, comments, and notifications. Converting a compact Connextra story into project-consistent Gherkin is consequently a constrained generation problem rather than a simple paraphrasing task.

1.1 Research Gap and Motivation

Prior work shows that LLMs can generate BDD artifacts, but existing evaluations differ substantially in data scale, model family, and validation method. Rathnayake et al. evaluated GPT-4, Claude 3, and Gemini on a proprietary BDD dataset and reported a BERTScore F1 of 91.16% for GPT-4 [1]. Karpurapu et al. reported fewer Gherkin-lint syntax errors under few-shot prompting than zero-shot prompting; across their evaluated models, zero-shot outputs accounted for 89% of the observed syntax errors [2]. Ferreira et al. evaluated a GPT-4 Turbo industrial acceptance-testing pipeline that generated Gherkin scenarios and downstream Cypress tests [3], while Fernandes et al. compared several LLMs on a much smaller public sample [4]. These studies establish feasibility, but they do not jointly answer whether a current compact model can produce both parser-ready and semantically close Gherkin across multiple public software domains.

This study addresses that narrower gap by evaluating the pinned gpt-5.4-mini-2026-03-17 snapshot on 261 stories from three unrelated open-source projects, using same-project one-shot prompting and two separate outcome measures. The emphasis is not that GPT-5.4-mini is inherently superior to earlier models. Rather, the contribution is an auditable evaluation design that separates syntactic validity from semantic similarity on a public, multi-domain benchmark, and that tests how far the similarity metric can be trusted. Because the stories were reverse-engineered from the reference files and the threshold was not calibrated, this revised version adds a leakage measurement, a defect audit of high-scoring outputs, a perturbation test of the metric, a threshold sensitivity analysis, per-project results and a prompt ablation.

1.2 Research Questions and Contributions

RQ1 – Skeleton similarity. Does the median skeleton cosine similarity between GPT-5.4-mini's generated files and their references exceed the proposal's threshold of 0.85?
RQ2 – Syntactic validity. Does GPT-5.4-mini generate feature files that the behave Gherkin parser accepts at a rate above 80%?

The paper makes three contributions: (1) a 261-case, multi-domain benchmark derived from expert-written Gherkin in Sylius, Apache Fineract, and Diaspora, with its construction and leakage risk documented; (2) an evaluation that reports

parser compliance separately from semantic similarity and checks the similarity metric against defect labels, seeded defects and alternative thresholds; and (3) a replication package with the pinned model snapshot, prompts, scripts, API logs, outputs and all revision analyses.

2 Related Work

2.1 LLM-Based Gherkin and BDD Generation

Research on LLM-assisted BDD generation has typically emphasized either semantic resemblance or syntactic correctness. Rathnayake et al. [1] evaluated multiple frontier models on 500 proprietary stories and reported strong BERTScore performance. The scale is useful, but the private dataset limits independent replication. Fernandes et al. [4] used public material and compared seven LLMs, but their evaluation comprised only ten test descriptions. Karpurapu et al. [2] evaluated GPT-3.5, GPT-4, Llama-2-13B, and PaLM-2 and found fewer Gherkin-lint syntax errors with few-shot prompting overall; zero-shot outputs accounted for 89% of the observed syntax errors. This supports evaluating example-based prompting, while noting that their validator was a linter rather than a runtime parser.

Industrial pipeline studies provide a complementary view. Ferreira et al. [3] evaluated GPT-4 Turbo in an industrial web-application workflow; their downstream Test Flow evaluation produced 50 Cypress test cases and reported 100% TypeScript syntactic correctness. Fonseca et al. [5] evaluated the AToMIC framework on 13 issues from the BMW MyBMW application and reported 93.3% syntactically correct generated Gherkin scenarios. These results demonstrate that LLMs can be integrated into practical workflows, but single-organization or single-application evaluations make cross-domain generalization difficult to assess.

2.2 Test Generation and Semantic Evaluation

Broader software-test generation research shows that syntactic success and fault-revealing or semantic quality should not be conflated. TestBench provides a benchmark for evaluating class-level test-case generation across LLMs [6]. Chang and Shokrolah Shirazi propose a systematic benchmark for assessing LLM test-case generation over generated program structures [7]. XUAT-Copilot uses multiple LLM-based agents for automated user-acceptance testing of WeChat Pay [8]. These systems solve different tasks from Gherkin generation, but they reinforce a common methodological lesson: generation quality is multi-dimensional, and an apparently successful artifact may still be weak on the property that matters to users.

For semantic comparison, sentence embeddings provide a scalable alternative to manual labeling. Sentence-BERT [9] enables dense representations suitable for cosine similarity. However, concrete names and values in raw Gherkin can dominate textual resemblance without representing domain intent. The present

study therefore masks quoted values and numbers and keeps only keyword lines before embedding, and it treats the resulting skeleton similarity as a proxy rather than a definitive measure of business correctness. Section 4.4 tests how well that proxy tracks business-logic defects.

2.3 Positioning of This Study

Compared with prior BDD-generation studies, this work combines four design choices in one experiment: a public multi-domain dataset, a pinned compact GPT model, same-project one-shot prompting, and separate parser and semantic evaluation. The contribution is therefore methodological and empirical rather than a claim of model novelty. The study also avoids interpreting a parser pass as semantic correctness: the behave parser answers only whether the generated file is valid Gherkin, while skeleton cosine similarity estimates closeness to the expert reference. These outcomes are reported and interpreted separately.

3 Methodology

3.1 Dataset Construction

The dataset pairs 261 expert-written .feature files from three open-source projects with 261 user stories. The stories were not collected from a backlog. They were reverse-engineered from the feature files: GPT-5.4-mini (temperature 0) received the text of each feature file and was asked to write one Connextra-style story ("As a [role], I want [feature], so that [benefit]") with as many acceptance criteria as needed to cover all of its scenarios. Only the first 15,000 characters of each file were passed to the model. This truncated 84 of the 261 files, all of them from Apache Fineract (84 of its 100 files, whose median length is about 30,000 characters). The full feature file is always used as the reference. Section 6 discusses the leakage risk this construction creates, and Section 4.5 measures it.

The unit of analysis is a feature file, not a scenario. With the behave 1.3.3 parser, the 258 analysed reference files contain 2,554 expanded scenarios and the corresponding generated files contain 1,913. The reference files were not screened with a parser before inclusion: 14 of the 261 reference files fail to parse with behave 1.3.3 (13 with behave 1.2.6). They remain in the dataset, because the metric of Section 3.4 does not require a parse.

The projects were chosen for domain and style diversity. Sylius covers products, carts, channels, shipping and taxes. Fineract contains data-heavy loan workflows with dates, balances, repayment schedules and data tables. Diaspora covers social-network behaviour driven through the browser. The benchmark is a deliberately heterogeneous sample of mature public English-language repositories, not a representative sample of BDD practice.

3.2 Model and Prompt Configuration

All generations use the snapshot gpt-5.4-mini-2026-03-17 through the OpenAI Chat Completions API with temperature 0. The requests set no maximum-token

Table 1. Dataset composition. The three exemplars of the original prompt (one per project) are excluded from the primary analysis.

Project	Domain	Feature files	Analysed
Sylius	E-commerce	100	99
Apache Fineract	Finance / banking	100	99
Diaspora	Social network	61	60
Total		261	258

cap, which deviates from the research proposal's planned cap of 1,500 output tokens. The original condition (C0) is same-project one-shot prompting: each project has its own system message and one example pair (user story and expert Gherkin) from the same project (Listing 1 in Appendix A). The three example files (ids 2, 102 and 202) are also members of the dataset, and their own generated files score 1.000, 0.997 and 1.000 on the metric below. The primary analysis therefore excludes them ($N = 258$); results for all 261 files are reported alongside.

Prompt ablation (added in revision). Two further conditions test how much the results depend on the example. C1 is zero-shot: the same system message and wording, with the example block and the sentence referring to it removed. C2 is one-shot with a different exemplar per project (ids 86, 119 and 203), drawn with a fixed random seed (2026) from the remaining files of each project before any C2 output existed. Model, temperature, system messages and output clean-up are identical in all conditions. The ablation is analysed on the 255 files that are not an exemplar in any condition. Its analysis plan was written before the C1 and C2 runs and is included in the replication package with its SHA-256 hash. It specifies paired Wilcoxon signed-rank tests on per-file cosine and exact McNemar tests on parser outcomes, each against C0 and Holm-corrected. The C1 and C2 runs took place on 16 October 2026. Because the environment used for this analysis could not download the encoder, skeleton cosine for all three conditions was computed with an ONNX export of all-MiniLM-L6-v2 using the same tokenisation, 256-token truncation, mean pooling and normalisation; it reproduces the original C0 scores to within 4×10^{-7}, and a rerun with sentence-transformers (model revision `1110a24`) agrees with it to within 3×10^{-7} in all three conditions.

3.3 Generation and Recording Pipeline

The pipeline loads the stories, builds the prompt for the story's project and calls the API. It removes a Markdown code fence only when the model wraps its output in one; no generated text is edited by hand. Failed requests are retried for the same story. In C0 all 261 requests returned an output, so no file was dropped. Each call is appended to a log with its timestamp and token counts. In the C0 log, the model field is the requested model name written by the script, not the name returned by the API, and the logged cost was computed with outdated prices. Recomputed from the logged token counts (328,257 prompt and

380,494 completion tokens) at the Standard list price of $0.75 and $4.50 per million input and output tokens, without cache discounts, the estimated C0 cost is approximately $1.96 (an estimate from the logged tokens, not an invoice). The C1 and C2 runs log the model name returned by the API.

3.4 Metric 1: Skeleton Cosine Similarity

Skeleton cosine similarity compares a generated file with its reference after reducing both to a skeleton. The skeleton keeps every line that begins with a Gherkin keyword (Feature, Background, Scenario, Given, When, Then, And, But), including the keyword itself. It removes tags, replaces every double-quoted string with `<PARAM>` and every number with `<NUM>`, and drops all other lines, so data tables, doc strings and Examples tables disappear (Table 5 in Appendix A). Each skeleton is encoded as a whole with all-MiniLM-L6-v2 [9], which mean-pools token embeddings and truncates its input at 256 word-piece tokens. With g and r the embeddings of the generated and reference skeletons,

$$\mathrm{SkelCos}(g, r) = \frac{g \cdot r}{\|g\|\|r\|}. \tag{1}$$

Two properties of this design limit what the metric can show. First, masking makes the score blind to changed numbers and quoted values by construction. Second, truncation means that the score often reflects only the start of a file: 174 of the 258 reference skeletons (67.4%) and 166 of the 258 generated skeletons (64.3%) exceed 256 tokens, including 94 of the 99 Fineract references (median 3,506 tokens). Section 4.4 therefore checks the metric against defect evidence instead of treating it as a measure of business correctness.

The threshold of 0.85 was fixed in the research proposal before the full run. It was not calibrated against human judgement. No prior study reports skeleton cosine with this encoder for Gherkin, and the proposal's pilot (30 stories, ids 1–10, 101–110 and 201–210, mean 0.859) used stories that are also part of the evaluated set. We therefore treat 0.85 as an operational reference point and report how the conclusions change between 0.80 and 0.90 (Section 4.3).

3.5 Metric 2: Parser Compliance

Parser compliance is the share of generated files that the behave Gherkin parser accepts. For a generated set S,

$$\mathrm{ESR} = \frac{|\{s \in S \mid \mathrm{parse}(s) = \mathrm{PASS}\}|}{|S|}. \tag{2}$$

We keep the proposal's name, executable syntax rate (ESR), but the check is narrower than the name suggests. Each file is passed to behave's parser (behave 1.3.3); no step definitions exist, so no step is matched or executed. A PASS means only that the file is valid Gherkin. It does not mean that the scenarios are complete, executable against the application or correct. The submitted version

ran behave --dry-run for this check. Without a steps directory, behave stops with a configuration error before parsing the file, and the original script counted that outcome as a PASS. All parser results in this version were therefore recomputed with the parser itself; behave 1.2.6 and 1.3.3 agree on every generated file.

3.6 Defect Audit and Structural Checks (added in revision)

Because neither metric checks business intent, we audited high-scoring outputs for defects. Of the 258 analysed files, 136 have a skeleton cosine of at least 0.85. From these we drew 20 files per project with a fixed seed (2026), giving 60 files. Each file was labelled against its user story and acceptance criteria; the reference was consulted only for project conventions. For each criterion the labeller recorded two facets. The business-logic facet covers wrong or missing numbers, operators, negations and conditions, wrong expected outcomes, untested criteria and contradicting additions. The temporal/causal facet covers missing preconditions, wrong triggering actions, missing or non-causal outcomes and impossible state orderings. Each defect received a severity from 0 (none) to 3 (the test would pass on a system that violates the criterion), and a file counts as defective if it has at least one defect of severity 2 or higher. Line-level evidence was recorded for each defect.

The labels were drafted by an AI coding assistant (OpenAI Codex) and then reviewed by a second author, who retained all of them unchanged. That review was recorded as one confirmation covering all 60 files, not as a decision with reasons per file. To estimate reliability, the first author, who reports not having seen the labels or their aggregate rates before rating, labelled a subset of 15 files fixed in advance (6 Sylius, 3 Fineract, 6 Diaspora) with the same guide. We report Cohen's κ [10] between these independent labels and the reviewed labels. This is agreement between one independent human rater and AI-drafted, human-reviewed labels, not agreement between two independent human raters.

Separately, an automated check parses each file and flags four keyword-order violations per scenario: no Then step, a Given after a When or Then, a Then before any When, and a non-Given step in the Background. These flags encode the checker's conventions and do not by themselves establish a business defect.

Finally, a perturbation test applied eight defect-seeding operators to 120 scenarios extracted from the reference files and compared each mutant with its original using the same metric (Appendix B).

3.7 Statistical Analysis Plan

The confirmatory tests use $\alpha = 0.05$ and follow standard guidance on matching tests to data properties [11].

RQ1: one-sample Wilcoxon signed-rank test. The test is applied to $x_i - 0.85$, one-sided, with zero differences dropped. H_0: the median skeleton cosine is at most 0.85; H_1: it exceeds 0.85. The proposal phrased these hypotheses in terms of the mean; we state them for the median, which the signed-rank test addresses under the assumption that the differences are symmetric. As effect size

we report $\delta = P(x > 0.85) - P(x < 0.85)$, a one-sample analogue of Cliff's δ against the threshold. Quartiles use linear interpolation and the SD uses $n - 1$. A non-significant result is read as insufficient evidence for performance above the threshold, not as evidence of poor performance.

RQ2: exact binomial test. Each file has a binary parser outcome. We test H_0: $p_{\mathrm{ESR}} \leq 0.80$ against H_1: $p_{\mathrm{ESR}} > 0.80$ [12] and report a 95% Clopper–Pearson interval [13].

Analyses added in revision. The following analyses were planned after the reviews and are exploratory. Per-project differences are tested with a Kruskal–Wallis test [14], reported with $\eta^2_H = (H - k + 1)/(n - k)$ and followed by pairwise two-sided Mann–Whitney tests with Holm correction [15]. Threshold sensitivity repeats the RQ1 test for thresholds from 0.80 to 0.90. Lexical leakage is measured as the share of distinct word trigrams of the reference file that also occur in the user story, and is related to skeleton cosine with Spearman's ρ.

3.8 Reproducibility and Denominators

The replication package contains the dataset, prompts, scripts, generated outputs, API logs and all revision analyses at https://github.com/simonhoang611/RT-SWT-004-Group4. The primary analysis uses $N = 258$ files, and no generated file was excluded because of its content; only the three exemplars are left out of the primary analysis.

4 Results

4.1 RQ1: Skeleton Similarity

RQ1 asks whether the median skeleton cosine similarity of C0 exceeds 0.85. On the 258 non-exemplar files the median is 0.853, 0.003 above the threshold, and the one-sided Wilcoxon test gives $p = 0.474$ with $\delta = 0.054$ (Table 2; Fig. 1 in Appendix A). Including the three exemplars gives $p = 0.355$. Decision: fail to reject H_0 in both analyses.

4.2 Results per Project

The pooled median hides large differences between projects (Table 3; Fig. 2 in Appendix A). Sylius is the only project above the threshold: its median is 0.887 and 77.8% of its files score at least 0.85. Fineract (median 0.826) and Diaspora (median 0.831) are significantly below 0.85 (one-sided $p < 0.001$ and $p = 0.008$). The projects differ (Kruskal–Wallis $H = 60.87$, df $= 2$, $p < 0.001$, $\eta^2_H = 0.231$). Sylius differs from Fineract and from Diaspora (Holm-adjusted $p < 0.001$ for both), while the difference between Fineract and Diaspora is not significant ($p = 0.74$). Spread is also uneven: the interquartile range is largest for Diaspora (0.087) and smallest for Fineract (0.062). The pooled result therefore does not describe a general capability: it averages one project above the threshold with two below it.

Table 2. Skeleton cosine similarity of condition C0. The primary analysis excludes the three prompt exemplars.

Statistic	$N = 258$ (primary)	$N = 261$
Minimum	0.585	0.585
Q1	0.805	0.806
Median	0.853	0.854
Q3	0.890	0.894
Maximum	0.994	1.000
Mean ± SD	0.848 ± 0.068	0.850 ± 0.069
Files ≥ 0.85	136 (52.7%)	139 (53.3%)
Wilcoxon W, one-sided p	16,784, 0.474	17,548, 0.355
$\delta = P(x > 0.85) - P(x < 0.85)$	0.054	0.065

Table 3. Skeleton cosine per project (C0, $N = 258$). The p-values are one-sided Wilcoxon tests against 0.85 in each direction.

Project	N	Median (IQR)	≥ 0.85 [95% CI]	$p_{>0.85}$	$p_{<0.85}$
Sylius	99	0.887 (0.076)	77.8% [68.3, 85.5]	< 0.001	1.000
Fineract	99	0.826 (0.062)	34.3% [25.1, 44.6]	1.000	< 0.001
Diaspora	60	0.831 (0.087)	41.7% [29.1, 55.1]	0.992	0.008
All	258	0.853 (0.085)	52.7% [46.4, 58.9]	0.474	0.526

4.3 Sensitivity to the Threshold

Because 0.85 was not calibrated, we repeated the RQ1 test for thresholds between 0.80 and 0.90 (Table 6 in Appendix A). The pooled median lies significantly above every threshold up to 0.84 ($p = 0.0097$ at 0.84) and above none from 0.85 upwards. The conclusion of RQ1 therefore depends on where the threshold is placed within a few hundredths. The project ranking does not: at every threshold Sylius has the largest share of files above it, and Fineract the smallest from 0.83 upwards.

We could not calibrate the threshold against defect labels. The defect audit (Section 4.4) sampled only files at or above 0.85, so it cannot estimate how often files below a threshold are defective. Within the audited range, cosine did not differ significantly between defective and non-defective files (Mann–Whitney $p = 0.21$; AUC 0.40 when a higher cosine is read as predicting a defect, i.e. defective files tended to score slightly lower). With 60 files from a restricted range, this neither shows nor rules out that the metric discriminates. Defects occurred in 11 of 24 files between 0.85 and 0.875, 10 of 16 between 0.875 and 0.90, 2 of 11 between 0.90 and 0.95, and 2 of 9 at 0.95 or above.

4.4 Defects in High-Similarity Outputs

Defect audit. Of the 60 audited files, all with skeleton cosine of at least 0.85, 25 (41.7%, 95% CI [29.1, 55.1]) contain at least one defect of severity 2 or 3: 6 of 20 Sylius files, 16 of 20 Fineract files and 3 of 20 Diaspora files. The two facets flag

exactly the same 25 files, so we report them as one result, not as two independent findings. The most frequent business-logic defects are wrong expected outcomes (20 files), missing or wrong conditions (9) and wrong numbers (8); no wrong operator or lost negation was found. On the temporal/causal facet, outcomes without a verifying Then are most frequent (19 files), followed by Then steps that do not follow from the When (9) and missing preconditions (8). The share of 41.7% describes a sample balanced across projects and restricted to high-scoring files; it is not an estimate of the defect rate of the whole corpus.

For example, a criterion in the story of Sylius file 76 expects a cart total of \$73.80 after "one taxed product" is removed from a cart holding three units of a \$100 product and two units of a \$30 product, both taxed at 23%. That total corresponds to removing the \$100 product entirely. The generated scenario removes a single unit and still expects \$73.80, although its own data imply \$319.80. The file scores 0.914, and both the reviewed labels and the independent rater mark it as defective.

Agreement. On the 15 doubly labelled files, the independent rater agreed with the reviewed labels on 12 files for each facet. Cohen's κ was 0.571 for the business-logic facet and 0.609 for the temporal/causal facet. Both values are below the threshold of 0.7 set in our protocol. Of the six disagreements across the two facets, five are Diaspora files that the independent rater marked as defective and the reviewed labels did not; the sixth is a Sylius file in the opposite direction. All three Fineract files received the same labels.

Structural checks. The keyword-order check flags 5 of the 255 parseable generated files and 40 of the 244 parseable reference files. Generated files thus follow the Given–When–Then order more consistently than the references. The audit shows, however, that correct order does not imply correct content.

Perturbation test. In the 623 valid scenario pairs, no seeded defect lowered skeleton cosine below 0.8995. Changed numbers and quoted values leave the skeleton identical by construction, and swapped or reordered steps score at least 0.954, although the keyword-order check flags all of them (Appendix B). These results show insensitivity at the scenario level and do not estimate a false-negative rate for whole files.

4.5 Lexical Leakage from Reverse Engineering

The user stories are short compared with the references: a median of 4.9% of the distinct word trigrams of a reference file also occur in its story (Sylius 8.3%, Fineract 1.6%, Diaspora 5.8%). Higher overlap goes with higher similarity in two of the three projects: Spearman's ρ between overlap and skeleton cosine is 0.29 for Sylius ($p = 0.003$), 0.33 for Diaspora ($p = 0.009$) and 0.11 for Fineract ($p = 0.30$). The pooled value ($\rho = 0.43$) is inflated by the project effect, since Sylius is high on both measures. These correlations are consistent with part of the similarity coming from wording that the story carries over from the reference, but they do not establish a causal effect. No independently written backlog was available to test this directly.

4.6 RQ2: Parser Compliance

RQ2 asks whether more than 80% of the generated files are valid Gherkin. Of the 258 non-exemplar files, 255 pass the parser (98.84%, 95% CI [96.64%, 99.76%], $p < 0.001$; Table 7 in Appendix A). The three failures (files 136, 148 and 154) are an Examples block outside a Scenario Outline in two files and a malformed data table in one. Decision: reject H_0. This corrects the 100% reported in the submitted version, which came from the faulty dry-run check described in Section 3; the decision is unchanged.

4.7 Prompt Ablation

All 261 requests of C1 and of C2 returned an output (Table 4; 255 common files). Removing the example (C1) lowered skeleton cosine in 182 of the 255 files (median paired difference -0.028; paired Wilcoxon, Holm-adjusted $p < 0.001$). The median fell from 0.853 to 0.819 and the share of files at or above 0.85 from 52.9% to 34.5%. The drop was largest in Fineract, where only 3 of 98 files (3.1%) reached 0.85.

Replacing the exemplars (C2) raised cosine in 145 files and lowered it in 110 (median paired difference $+0.008$; Holm-adjusted $p < 0.001$). Under C2 the median is 0.875 and lies significantly above 0.85 (one-sided $p < 0.001$), so a different choice of exemplars would reverse the RQ1 decision. Descriptively, almost all of this change comes from Fineract (median 0.827 under C0, 0.876 under C2), while Sylius and Diaspora change by less than 0.01.

Parser compliance was similar under all three prompts. Under C0, C1 and C2, 252, 252 and 254 of the 255 files parse (exact McNemar against C0, Holm-adjusted $p = 1.0$ for both); the tests do not establish equivalence. Outputs were longer under C1 and C2 than under C0 (median 764 and 772 completion tokens against 509).

4.8 Consolidated Answers

RQ1: not supported. The median similarity is not demonstrably above 0.85, only Sylius exceeds it, and the conclusion changes with the threshold within a few hundredths and with the choice of exemplars. RQ2: yes. Nearly all generated files are valid Gherkin, with or without an example in the prompt. High similarity does not guarantee correct content: 25 of 60 audited high-scoring files contain a business-logic defect, and seeded defects leave the metric almost unchanged.

5 Discussion

5.1 Why Parser Compliance and Semantic Similarity Diverge

Nearly all generated files are valid Gherkin (255 of 258), so the model reliably follows the surface grammar. Same-project examples may contribute by showing

Table 4. Prompt ablation on the 255 files that are not an exemplar in any condition. Skeleton cosine medians overall and per project (n = 98, 98, 59), share of files ≥ 0.85 with 95% CI, and parser PASS count.

Condition	Median	≥ 0.85 [95% CI]	Sylius	Fineract	Diaspora	PASS
C0 (exemplars 2, 102, 202)	0.853	52.9% [46.6, 59.2]	0.886	0.827	0.835	252
C1 zero-shot	0.819	34.5% [28.7, 40.7]	0.873	0.769	0.825	252
C2 (exemplars 86, 119, 203)	0.875	64.3% [58.1, 70.2]	0.892	0.876	0.840	254

keyword order, scenario structure, tables and local conventions, which is consistent with prior work showing that examples reduce syntax errors [2] and with industrial studies reporting high syntactic correctness [3,5]. The prompt ablation (Section 4.7) shows that this result does not require the example: zero-shot outputs parse as often as one-shot outputs (252 of 255 in both conditions), and alternative exemplars give 254. The McNemar tests found no difference, although they cannot establish equivalence.

Semantic preservation is harder, and the per-project results locate the difficulty. Fineract files are long, data-driven and full of dates, rates and balances; 84 of its 100 stories were written from a truncated reference, and 94 of its 99 analysed references are cut by the encoder's 256-token limit. Diaspora files depend on browser-level steps and selectors that a story does not state. Sylius checkout and cart scenarios are shorter and closer to their stories. The experiment does not isolate the causal contribution of each factor, but it shows that a single pooled median is not a meaningful summary of capability across these projects.

5.2 What the Similarity Metric Can and Cannot Show

The ablation also shows how fragile the RQ1 decision is. A different exemplar per project, drawn at random, moves the median from 0.853 to 0.875 and makes it significantly larger than 0.85, while removing the example lowers it to 0.819. Most of the change occurs in Fineract. We conjecture that this follows from truncation: the encoder sees only the first 256 tokens of 94 of the 99 Fineract references, so the opening of a file, that is its Background and first steps (the skeleton drops all table rows), largely determines the score, and an exemplar that shows a typical opening can raise the score without any evidence that the business content improved. The ablation does not test this explanation.

The revision analyses show that skeleton cosine is a measure of structural resemblance, not of correct behaviour. Within the high-similarity range, 25 of 60 audited files contain a defect that changes what the test checks, and in this restricted sample cosine did not rank defective files significantly differently from non-defective ones. Seeded defects at the scenario level, including dropped Then steps and reordered steps, leave the score at or above 0.8995, and changed numbers or values cannot change it at all because the skeleton masks them. A threshold on this metric, whether 0.85 or any value between 0.80 and 0.90, therefore cannot certify that a generated file preserves its acceptance criteria. We report 0.85 as a descriptive reference point for structural similarity, together with its sensitivity analysis, and do not interpret files above it as correct.

5.3 Practical Use in BDD Workflows

The evidence supports a bounded workflow: use GPT-5.4-mini to draft valid Gherkin, then have a reviewer check expected outcomes, conditions, data values and the coverage of each acceptance criterion. The audit suggests where review effort is needed most: wrong expected outcomes and missing verifying Then steps were the most frequent defects, and defective files were far more frequent in the data-heavy finance domain (16 of 20) than in the other two (6 and 3 of 20). The experiment does not justify promoting generated scenarios into CI/CD as authoritative acceptance tests. A parser pass establishes only that a file is valid Gherkin, and a high similarity score does not establish correctness.

5.4 Relationship to Prior Work

The findings refine, rather than contradict, earlier BDD-generation studies. High parser compliance aligns with the syntactic success reported in industrial pipelines [3,5] and with the lower syntax-error counts under few-shot prompting reported by Karpurapu et al. [2]. The semantic result is more cautious than studies reporting strong semantic or text-similarity results [1,4], partly because this experiment uses a different metric, a fixed threshold, a multi-domain sample and a hypothesis test against that threshold. Our defect audit also suggests that high embedding similarity alone, in any of these studies, says little about whether acceptance criteria are preserved. Cross-paper metric values are therefore contextual comparisons, not model rankings.

6 Threats to Validity

6.1 Internal Validity

Reverse-engineered stories. The user stories were written by GPT-5.4-mini from the reference files, so the input can carry wording, structure and scenario order from the very file it is compared with. The model that wrote the stories is also the model under test. Story–reference trigram overlap is low (median 4.9%) but correlates with similarity in two of the three projects (Section 4.5), which suggests that part of the measured similarity comes from this construction. Conversely, 84 Fineract stories were written from only the first 15,000 characters of a longer reference, so they cannot describe the rest of it, and the full reference is still used for comparison. Our trigram measure captures only lexical leakage, not structural leakage such as the order and number of criteria. No independently written backlog was available, so all results apply to reverse-engineered stories and may overstate performance on natural requirements.

Exemplars and pilot. The three original exemplars were part of the dataset; we exclude them from the primary analysis and report both denominators. The 30 pilot stories used when the threshold was set are part of the evaluated set.

Prompt ablation. C2 uses one random draw of exemplars; other draws may give other results. The ablation runs took place three months after C0, so a

change in the hosted service cannot be excluded, although the API returned the requested snapshot name for every C1 and C2 call; the C0 log records only the requested name.

Hosted model. The serving infrastructure is outside our control. Pinning the snapshot, using temperature 0 and logging each call reduce but do not remove nondeterminism. The C0 log records the requested rather than the returned model name.

6.2 Construct Validity

Similarity metric. Skeleton cosine is an imperfect proxy for business correctness. It is blind to numbers and quoted values by construction, ignores data tables and Examples, and for about two thirds of the files it sees only the first 256 tokens of a skeleton. Within the audited high-similarity files it did not separate defective from non-defective outputs significantly, and scenario-level seeded defects barely changed it. Conversely, a valid alternative scenario can score low because it differs from the single expert reference.

Defect labels. The 60 audit labels were drafted by an AI assistant and retained unchanged by a reviewing author, whose review was recorded as one confirmation rather than per-file decisions. Agreement with an independent human rater on 15 files (independence rests on the rater's own declaration) was $\kappa = 0.571$ (business logic) and $\kappa = 0.609$ (temporal/causal), below the 0.7 our protocol required. The protocol required a consensus round and relabelling in that case; we did not perform them, and we report all labels and both κ values unchanged. With 15 files κ is imprecise, and the disagreements suggest that the reviewed labels may undercount defects in the social-network domain. The defect rates should be read as indicative. The two label facets are not independent measurements, and the audit covers only files at or above 0.85.

Perturbation test. The test mutates single extracted scenarios, omitted the Background of 74 of 120 source scenarios, and draws several mutants from the same scenario, so its pairs are not independent and do not estimate whole-file sensitivity.

Parser compliance. A parser pass shows only that a file is valid Gherkin. It does not show successful execution against the application or correct step implementations. The submitted version's dry-run check did not measure even this, which we corrected.

6.3 External and Conclusion Validity

The dataset spans three mature, English-language public projects and is not representative of BDD practice. Results may differ for short tickets, mixed-language requirements, private conventions or projects with unstable step definitions. The projects contribute unequal numbers of files (99, 99 and 60 after exclusion), and they differ significantly in similarity, so pooled results do not generalise across domains.

Neither threshold is an industry standard. The 0.85 similarity threshold was fixed in the proposal without calibration and the RQ1 conclusion changes between 0.84 and 0.85. All analyses added in revision were planned after the reviews and are exploratory; their p-values are not corrected across analyses. The proposal planned a 1,500-token output cap that the runs did not apply, which may have affected output length, cost and both outcome measures; no capped run was made for comparison.

7 Conclusion

This study evaluated GPT-5.4-mini for Gherkin generation on Connextra-style user stories reverse-engineered from 261 feature files of three open-source projects, with the three prompt exemplars excluded from the primary analysis ($N = 258$). The two research questions produce different answers. For RQ1, the median skeleton cosine similarity was 0.853 against a threshold of 0.85, and the one-sided Wilcoxon test gave $p = 0.474$ ($\delta = 0.054$; $p = 0.355$ with all 261 files). We fail to reject H_0 and do not claim that GPT-5.4-mini exceeds the threshold. Only the e-commerce project exceeded it, while the finance and social-network projects lay significantly below it, and the conclusion would change at a threshold of 0.84. For RQ2, 255 of 258 generated files were valid Gherkin (98.84%, 95% CI [96.64%, 99.76%], $p < 0.001$), so we reject H_0. This corrects the 100% of the submitted version.

The revision analyses limit what the similarity results mean. In a sample of 60 files at or above the threshold, 25 contained a business-logic defect, and cosine did not separate defective from non-defective files significantly. Seeded defects left the score almost unchanged. These labels are AI-drafted and human-reviewed, and their agreement with an independent rater was below our protocol's threshold of 0.7 ($\kappa = 0.571$ and 0.609). A prompt ablation shows that the RQ1 decision also depends on the exemplars: with a different randomly drawn exemplar per project the median rises to 0.875 and exceeds 0.85 significantly, mostly because of Fineract, and without an example it falls to 0.819. Parser compliance stayed high under all three prompts (252 to 254 of 255 files).

For practice, the supported use case is assisted drafting: a QA engineer can start from a valid feature file and focus review on expected outcomes, conditions, data values and criterion coverage. Neither a parser pass nor a high similarity score supports autonomous acceptance. Future work should (1) use independently written requirements, (2) validate semantics with criterion-level review by independent raters or with execution against step definitions, and (3) compare retrieval-augmented and open-source baselines on the same files.

Data, Tool, and Ethics Disclosure

This study involves no human participants; the empirical subjects are public software artifacts and model-generated Gherkin scenarios. Human-subjects review is therefore not applicable. GPT-5.4-mini is the system under evaluation and

was also used to write the user stories from the reference files (Section 3). In the revision, an AI coding assistant (OpenAI Codex) drafted the 60 defect-audit labels, which an author reviewed, and AI assistants (OpenAI Codex and Anthropic Claude) helped write analysis scripts and draft text; Claude also prepared reading aids for the 15-file rater (rubric notes, an acceptance-criteria map, and reading sheets and a glossary for Fineract) that contained no labels; all numbers were recomputed from the released data and scripts, and the authors take responsibility for the content. The replication package contains the dataset, prompts, scripts, generated outputs, API logs, audit labels and revision analyses at https://github.com/simonhoang611/RT-SWT-004-Group4. The authors declare no external funding and no conflicts of interest.

References

1. Rathnayake, A., Shahin, M., Abaei, G.: Behaviour Driven Development Scenario Generation with Large Language Models. arXiv:2603.04729 (2026). https://doi.org/10.48550/arXiv.2603.04729
2. Karpurapu, S., Myneni, S., Nettur, U., Gajja, L.S., Burke, D., Stiehm, T., Payne, J.: Comprehensive Evaluation and Insights Into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation. IEEE Access 12 (2024). https://doi.org/10.1109/ACCESS.2024.3391815
3. Ferreira, M., Viegas, L., Faria, J.P., Lima, B.: Acceptance Test Generation with Large Language Models: An Industrial Case Study. In: IEEE/ACM International Workshop on Automation of Software Test (AST) (2025). arXiv:2504.07244
4. Fernandes, H., Perkusich, M., Albuquerque, D., Silva, I., Santos, D.F.S., Gorgonio, K., Perkusich, A.: A Comparative Study of LLMs for Gherkin Generation. In: Brazilian Symposium on Software Engineering (SBES), pp. 171–181 (2025). https://doi.org/10.5753/SBES.2025.9888
5. Fonseca, P.L., Lima, B., Faria, J.P.: Streamlining Acceptance Test Generation for Mobile Applications Through Large Language Models: An Industrial Case Study. In: 40th IEEE/ACM International Conference on Automated Software Engineering (ASE) (2025). https://doi.org/10.1109/ASE63991.2025.00273
6. Zhang, Q., Shang, Y., Fang, C., Gu, S., Yu, S., Zhou, J., Chen, Z.: TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models. Frontiers of Computer Science 21(6), 2106202 (2027), published online 13 October 2025. https://doi.org/10.1007/s11704-025-50078-9
7. Chang, H.-F., Shokrolah Shirazi, M.: A Systematic Approach for Assessing Large Language Models' Test Case Generation Capability. Software 4(1), 5 (2025). https://doi.org/10.3390/software4010005
8. Wang, Z., Wang, W., Li, Z., Wang, L., Yi, C., Xu, X., Cao, L., Su, H., Chen, S., Zhou, J.: XUAT-Copilot: Multi-Agent Collaborative System for Automated User Acceptance Testing with Large Language Model. arXiv:2401.02705 (2024). https://doi.org/10.48550/arXiv.2401.02705
9. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: EMNLP-IJCNLP, pp. 3982–3992 (2019). https://doi.org/10.18653/v1/D19-1410
10. Cohen, J.: A Coefficient of Agreement for Nominal Scales. Educational and Psychological Measurement 20(1), 37–46 (1960). https://doi.org/10.1177/001316446002000104

11. Arcuri, A., Briand, L.C.: A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In: 33rd International Conference on Software Engineering, pp. 1–10 (2011). https://doi.org/10.1145/1985793.1985795
12. Agresti, A.: Categorical Data Analysis, 2nd edn. John Wiley & Sons (2002). https://doi.org/10.1002/0471249688
13. Clopper, C.J., Pearson, E.S.: The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. Biometrika 26(4), 404–413 (1934). https://doi.org/10.1093/biomet/26.4.404
14. Kruskal, W.H., Wallis, W.A.: Use of Ranks in One-Criterion Variance Analysis. Journal of the American Statistical Association 47(260), 583–621 (1952). https://doi.org/10.1080/01621459.1952.10483441
15. Holm, S.: A Simple Sequentially Rejective Multiple Test Procedure. Scandinavian Journal of Statistics 6(2), 65–70 (1979). https://www.jstor.org/stable/4615733

A Prompt, Skeleton Example and Additional Results

Listing 1 gives the prompt template of condition C0 (Section 3), Table 5 illustrates the skeleton extraction of Section 3.4, Fig. 1 and Fig. 2 show the distribution of skeleton cosine in C0 overall and per project, Table 6 gives the threshold sensitivity analysis of Section 4.3, and Table 7 gives the parser results of RQ2.

Listing 1. Prompt template of condition C0. Braced placeholders are instantiated by project at run time.

```
System: {domain_specific_system_message}
User:
Generate a BDD Gherkin file for the following user story.
Follow the exact style, wording, and conventions shown in the Example.

EXAMPLE USER STORY:
{example_user_story_from_same_domain}

EXAMPLE GHERKIN:
{example_gherkin_from_same_domain}

---
NOW GENERATE FOR THIS USER STORY:
{user_story_text}
```

Table 5. Skeleton extraction on a short Sylius-style fragment.

Original line	Skeleton line
`@ui`	(removed)
`Given a product "T-Shirt" priced at "$20.00"`	`Given a product <PARAM> priced at <PARAM>`
`When I add 3 units to the cart`	`When I add <NUM> units to the cart`
`| name | price |`	(removed)
`Then the total should be "$60.00"`	`Then the total should be <PARAM>`

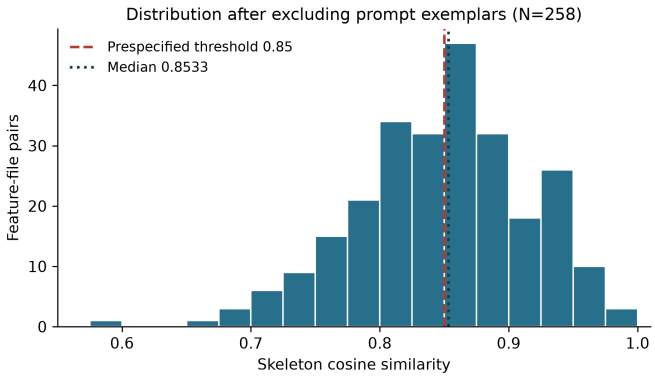


Fig. 1. Distribution of skeleton cosine similarity for the 258 non-exemplar files (condition C0).

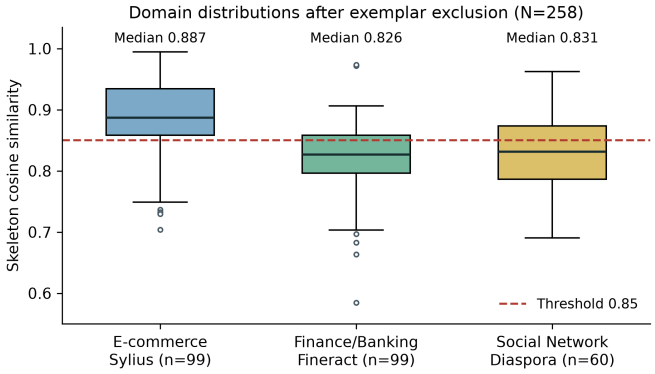


Fig. 2. Skeleton cosine similarity per project (C0, $N = 258$). The dashed line marks 0.85.

Table 6. Threshold sensitivity (C0, $N = 258$). The p-value is the one-sided Wilcoxon test that the median exceeds t.

t	Share $\geq t$ [95% CI]	Median $-t$	p	Sylius	Fineract	Diaspora
0.80	78.3% [72.8, 83.2]	+0.053	< 0.001	91.9%	73.7%	63.3%
0.82	67.8% [61.8, 73.5]	+0.033	< 0.001	87.9%	56.6%	53.3%
0.84	59.3% [53.0, 65.4]	+0.013	0.010	84.8%	40.4%	48.3%
0.85	52.7% [46.4, 58.9]	+0.003	0.474	77.8%	34.3%	41.7%
0.86	43.4% [37.3, 49.7]	−0.007	0.988	73.7%	22.2%	28.3%
0.88	30.6% [25.1, 36.6]	−0.027	1.000	55.6%	12.1%	20.0%
0.90	22.1% [17.2, 27.7]	−0.047	1.000	45.5%	4.0%	13.3%

Table 7. Parser compliance of condition C0 (behave 1.3.3 parser).

Statistic	$N = 258$ (primary)	$N = 261$
Parser PASS / FAIL	255 / 3	258 / 3
ESR	98.84%	98.85%
95% Clopper–Pearson CI	[96.64%, 99.76%]	[96.68%, 99.76%]
Exact binomial p ($p_0 = 0.80$)	4.6×10^{-21}	2.4×10^{-21}

B Perturbation Test

Design. The perturbation test measured whether the metric reacts to seeded defects. From the reference files, 120 scenarios (40 per project, fixed seed) were extracted, and eight operators were applied where applicable: changing a number, a quoted value, a comparison operator or a negation, dropping a Then or a Given step, swapping a When with a Then, and moving a Then to the front. Each mutant was compared with its original scenario using the same skeleton metric.

Results. The 120 extracted scenarios yielded 683 applicable mutants. Of these, 29 mutants do not parse, and the extraction omitted the Background of 74 of the 120 source scenarios. Restricting to the 623 pairs in which both texts parse and the original has no keyword-order flag, every mutant keeps a skeleton cosine of at least 0.8995 with its original. At 0.90, 622 of 623 mutants would still pass, and at 0.95, 97.9%. For changed numbers and quoted values (179 pairs) the skeletons are identical, which follows from masking and is not an empirical finding. Swapped When/Then steps and Then steps moved to the front (210 pairs) score at least 0.954, although the keyword-order check flags all of them. Because the test compares single scenarios and the main metric compares whole files, these results show insensitivity at the scenario level and do not estimate a false-negative rate for whole files.